NUS National University of Singapore

BT5110
Data Management and Warehousing

Entity-Relationship Model

DDL and DML

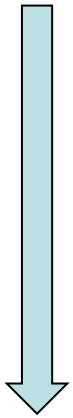
DDL

- Create table to implement schema
- Specify integrity constraints

DML

- Insert into table
- Update rows
- Delete data

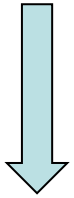
Movies (id: **INT**, title: **TEXT**,
genre: **TEXT**, opened: **DATE**)



```
CREATE TABLE Movies (
  id      INT PRIMARY KEY,
  title   TEXT NOT NULL,
  genre   TEXT NOT NULL,
  releaseYr INT NOT NULL
);
```

Table "Movies"

id	title	genre	opened



```
INSERT INTO Movies VALUES
  (101, 'Aliens', 'action', 1986),
  (102, 'Logan' , 'drama' , 2017);
```

Table "Movies"

id	title	genre	opened
101	Aliens	action	1986
102	Logan	drama	2017

Missing Step?

Open Questions

- Where does the schema come from?
- What tables do we need?
- What attributes should each table contain?
- What data integrity constraints are required?
- What are the table names, attribute names, data types?
- ***Which design one is better?***

Movies (id: **INT**, title: **TEXT**, genre: **TEXT**, releaseYr: **INT**)

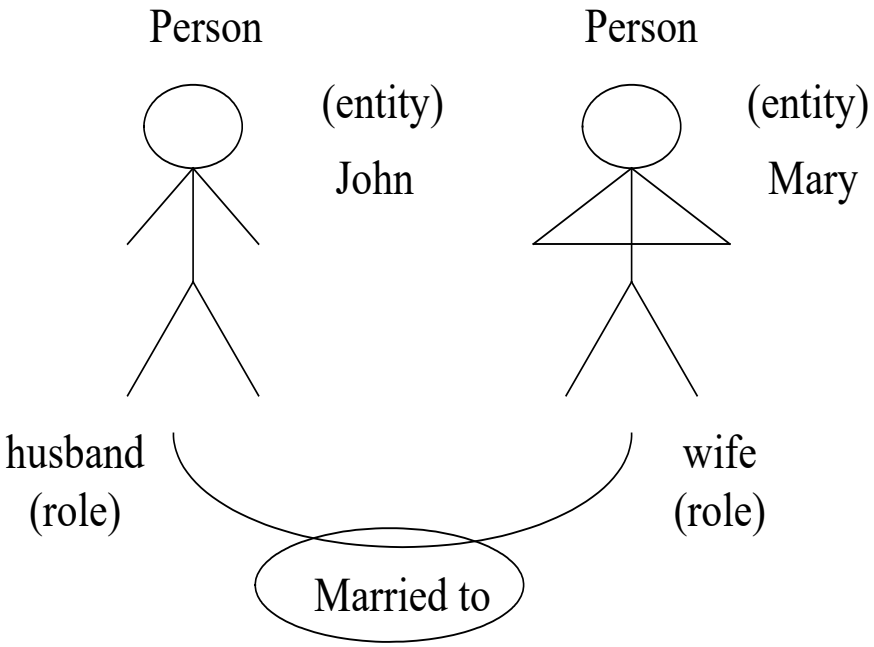
```
CREATE TABLE Movies (
  id        INT PRIMARY KEY,
  title     TEXT NOT NULL,
  genre     TEXT NOT NULL,
  releaseYr INT NOT NULL
);
```

```
CREATE TABLE Movies (
  title     TEXT,
  genre     TEXT NOT NULL,
  releaseYr INT,
  PRIMARY KEY (title, releaseYr)
);
```

Real World → Conceptual database design → CREATE TABLE

ER Model Basics

- The ER model was proposed by Prof. Peter Chen in 1976
- Main concepts:
  - entity (or object)
  - relationship
- English correspondence
  - noun → entity
  - verb → relationship



Requirement Analysis

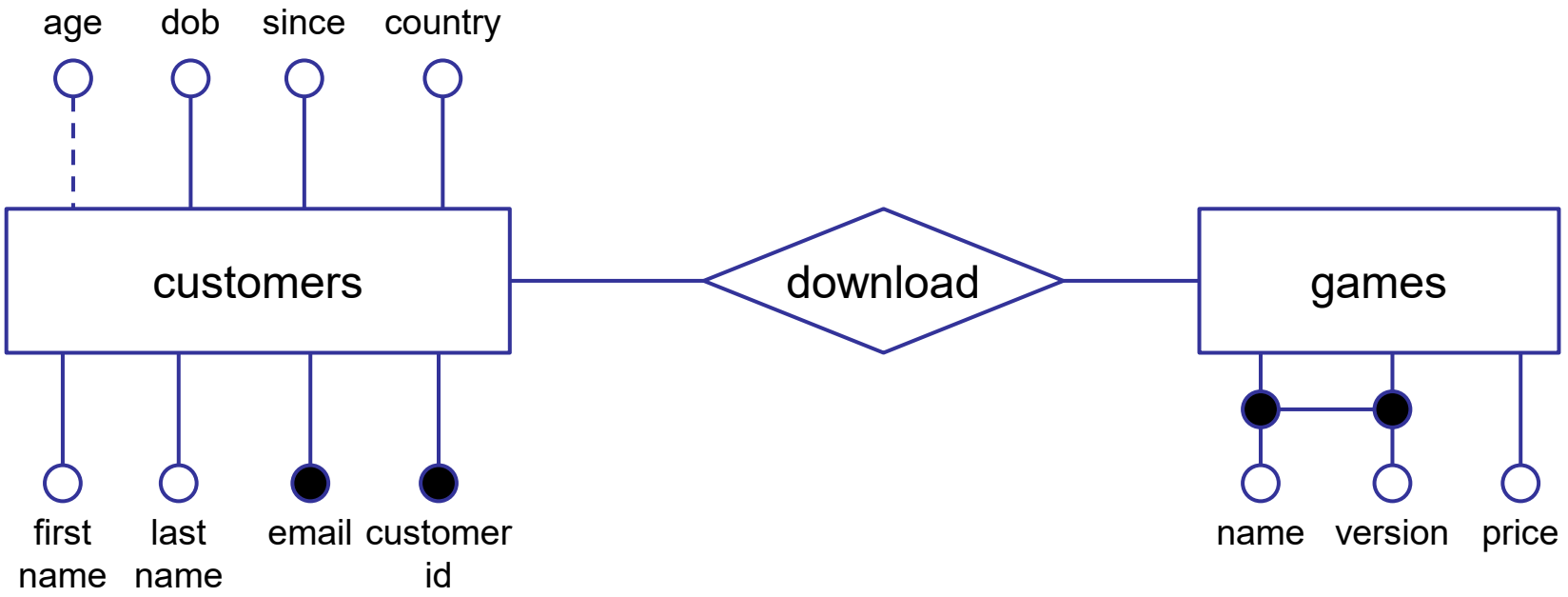
Game Store

Our company, Apasaja Pte Ltd, has been commissioned to develop an application to manage the data of an online app store. We want to store several items of information about our **customers** *such as their* ***first name****,* ***last name****,* ***age****,* ***e-mail****,* ***date****, and* ***country of registration*** *to our online sales service and the* ***customer identifier*** *that they have chosen.*

We also want to manage the list of our products, **games**, *their* ***name****, their* ***version****, and their* ***price****. The price is fixed for each version of each game. Finally, our customers buy and* **download** *games. We record which version of which game each customer has downloaded. It is not essential to keep the download date for this application.*

ER Diagram

- A **rectangle** represents an entity type
- A **diamond** represents a relationship set
- A **circle** represents an attribute

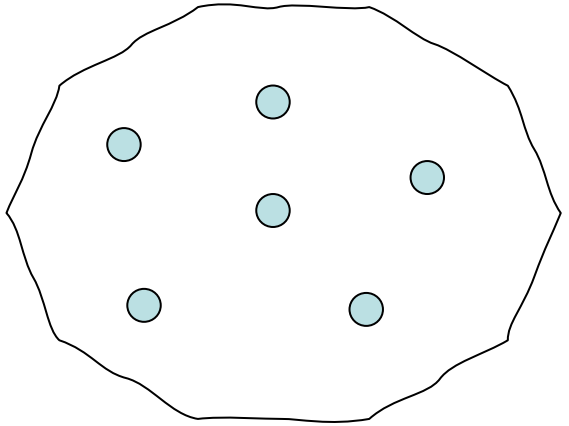


ER Diagram

Entities

- Entities are **distinguishable real-world objects** that we want to store information about.
- A **named rectangle** represents **an entity type**
- An **entity set** is the collection of all entities of that entity type currently stored in the database



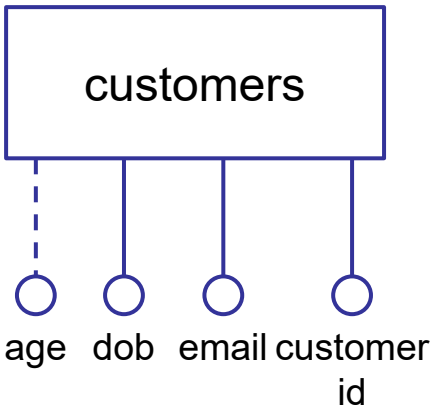


customers

ER Diagram

Attributes

- Every entity has **attributes**.
- All entities of the same entity type have the **same set of attributes**.
- Values of attributes differ from one entity to another
- Some attributes can be **derived** from other attributes

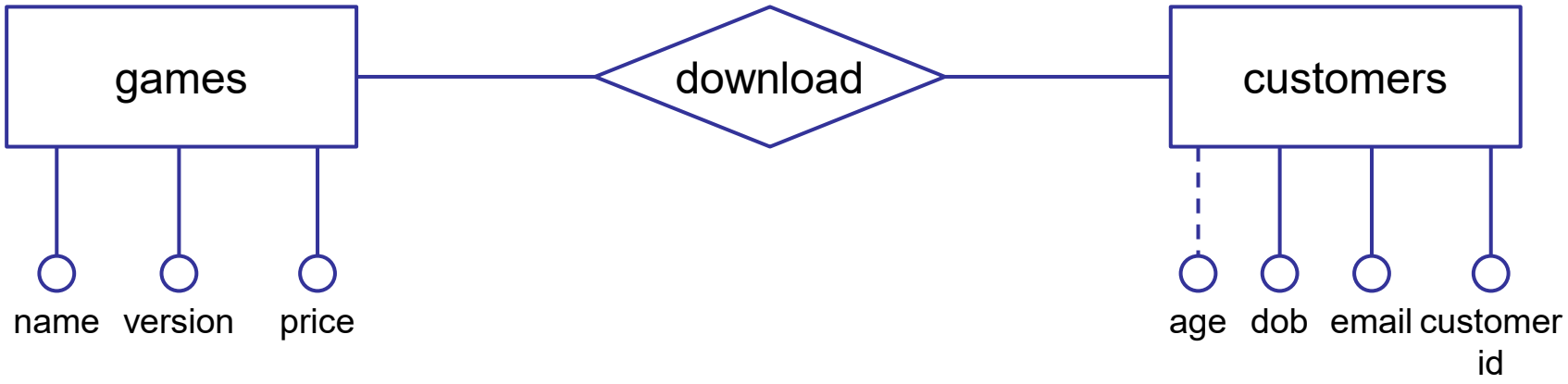


The ER model is **value-oriented**. Attribute values are atomic values such as strings, numbers or dates.

ER Diagram

Relationships

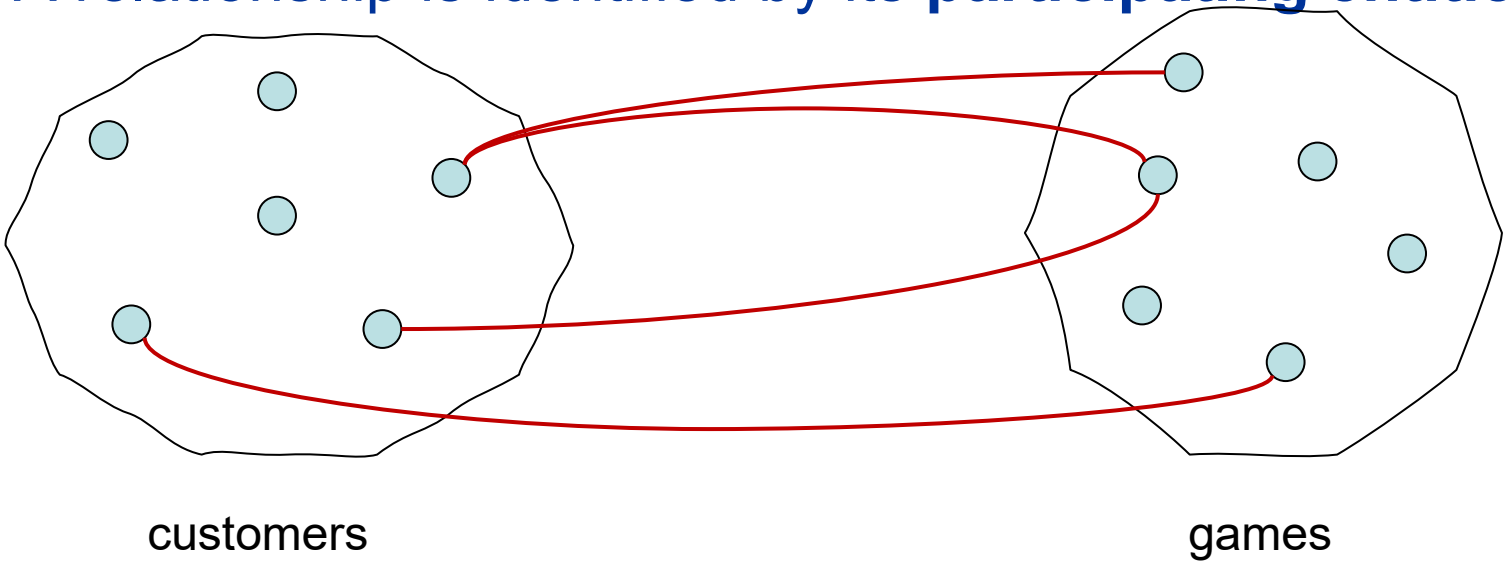
- A relationship **associates two or more entities**
- A **named diamond** represents a **relationship type**
- A relationship set is the collection of all relationship instances of that type currently stored in the database
- A relationship is identified by its **participating entities**



ER Diagram

Relationships

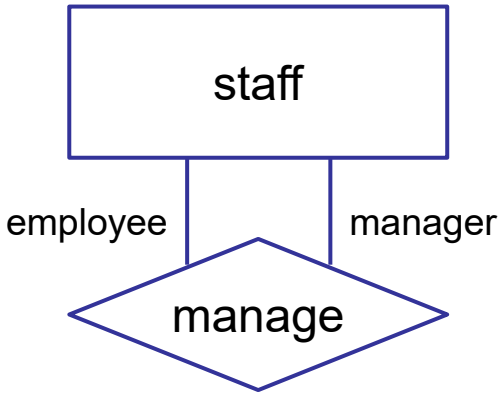
- A relationship **associates two or more entities**
- A **named diamond** represents a **relationship type**
- A relationship set is the collection of all relationship instances of that type currently stored in the database
- A relationship is identified by its **participating entities**

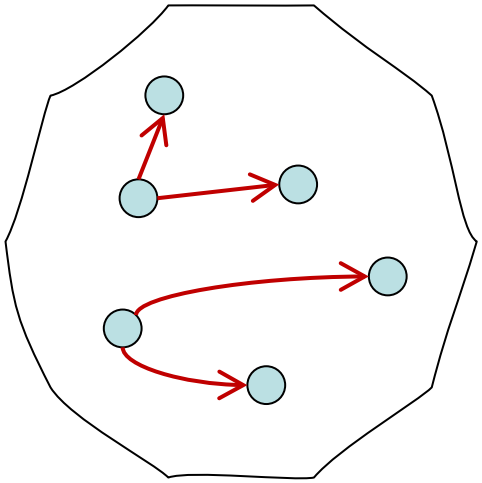


ER Diagram

Relationships

- Relationships can associate entities from the **same entity set**
- **Participation** can be **named** in general (**role name**)



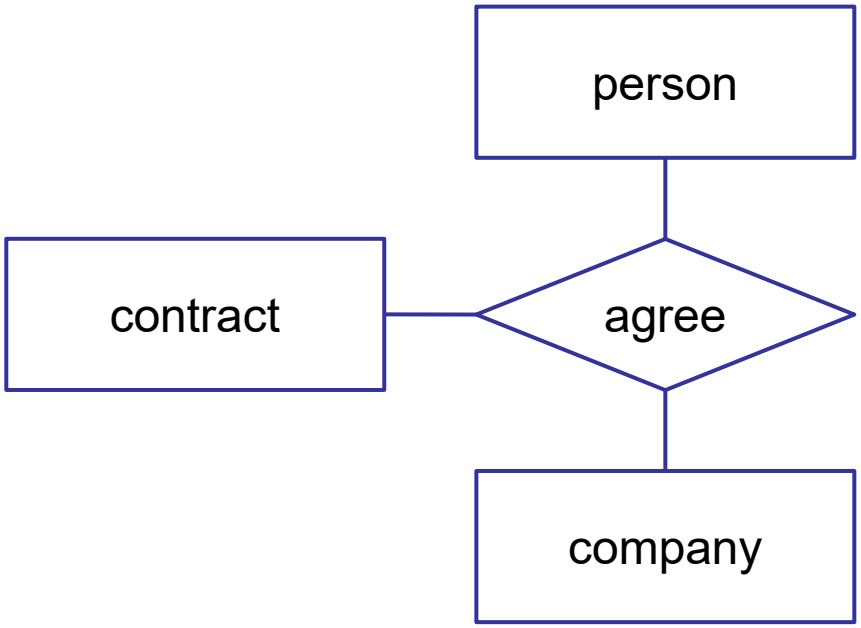


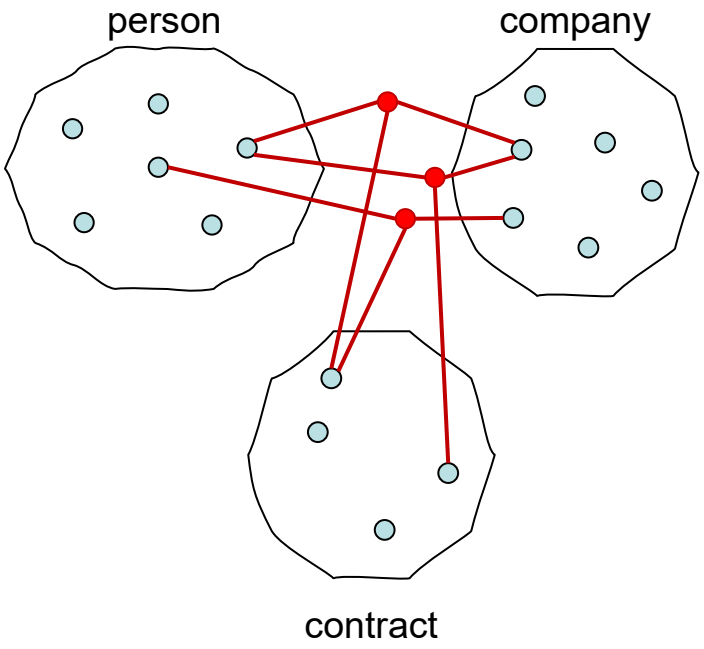
staff

ER Diagram

n-Ary Relationships

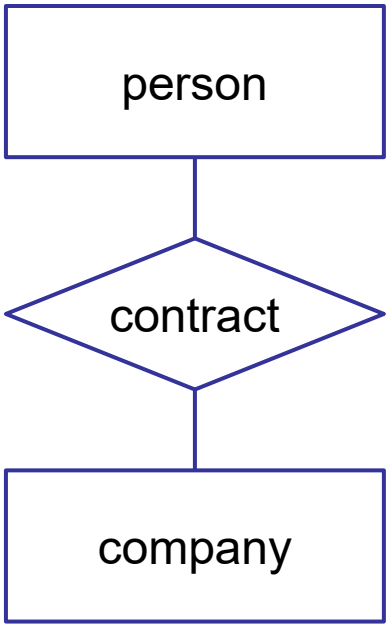
- Relationships can associate **more than two entity types**
- We call this an **n-ary** relationship.



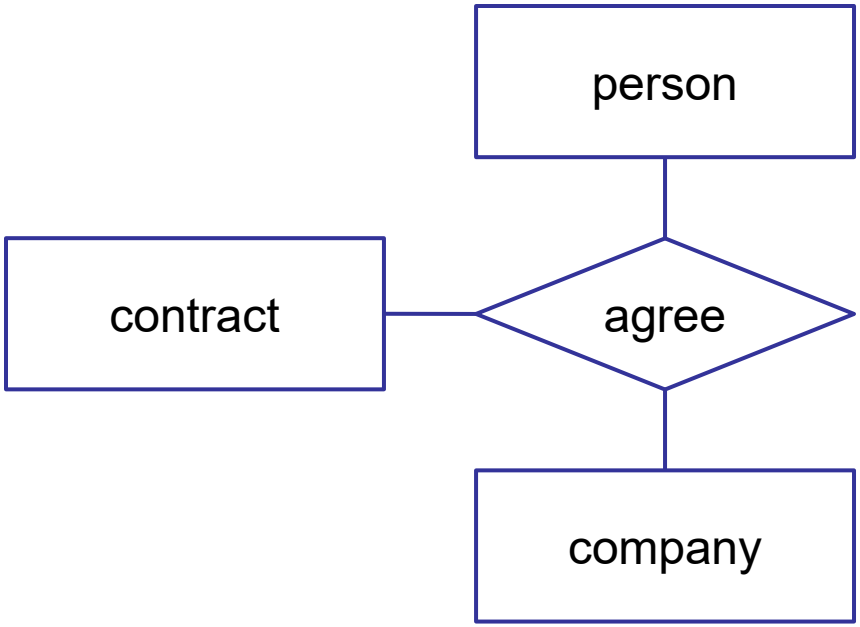


Binary vs n-ary Relationships

the same person **cannot** have two different contracts with the same company

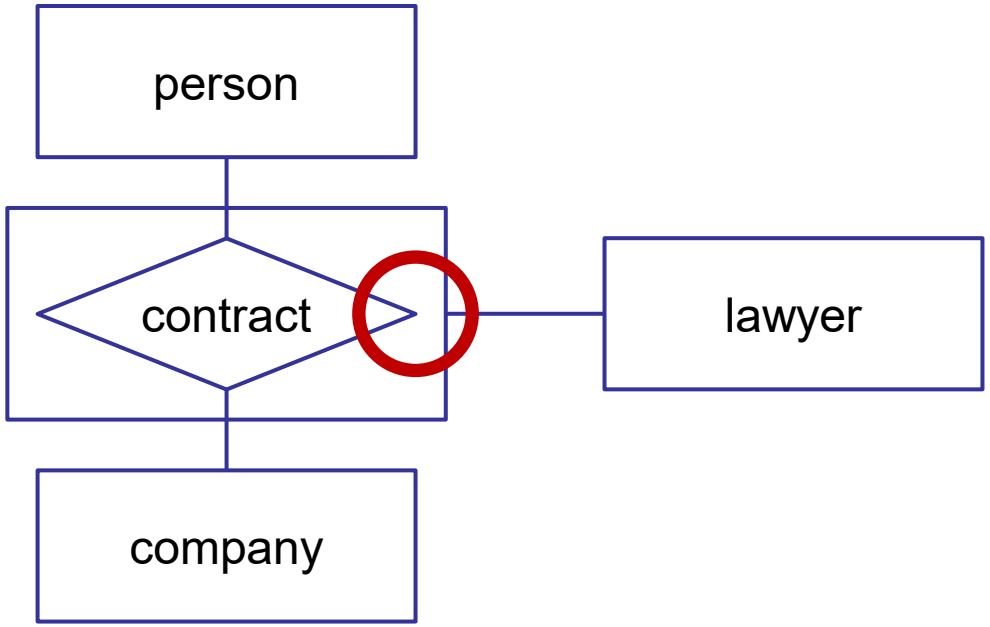


This is allowed in n-ary relationship



Aggregation

- Sometimes, we need to model a relationship involving another relationship
- We represent this by **wrapping** the relationship type in a **box**.

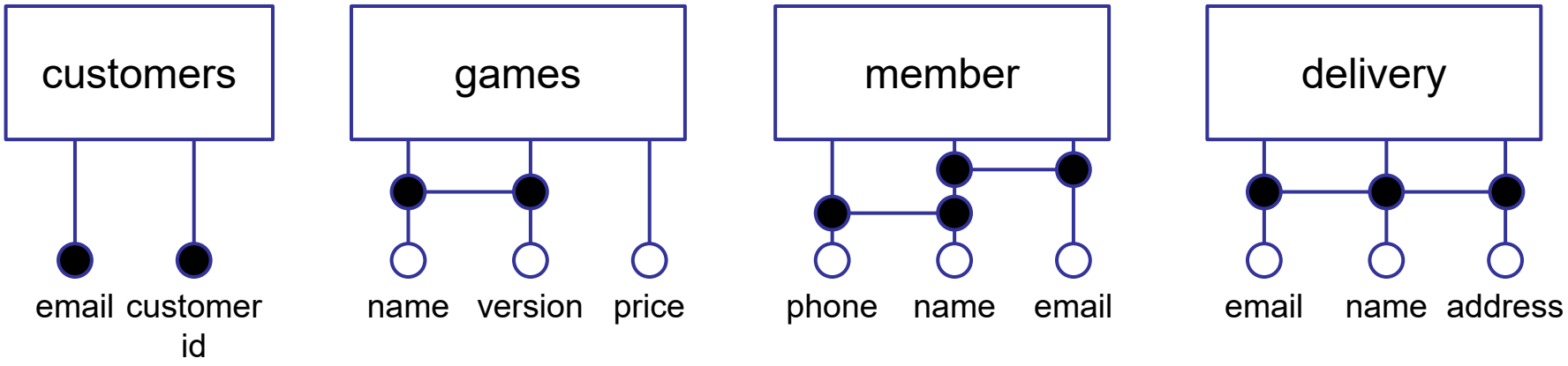


An **aggregate** is still a **relationship type**.
All the restrictions of a relationship type applies

The **gap** between diamond and box is important to avoid ambiguity.

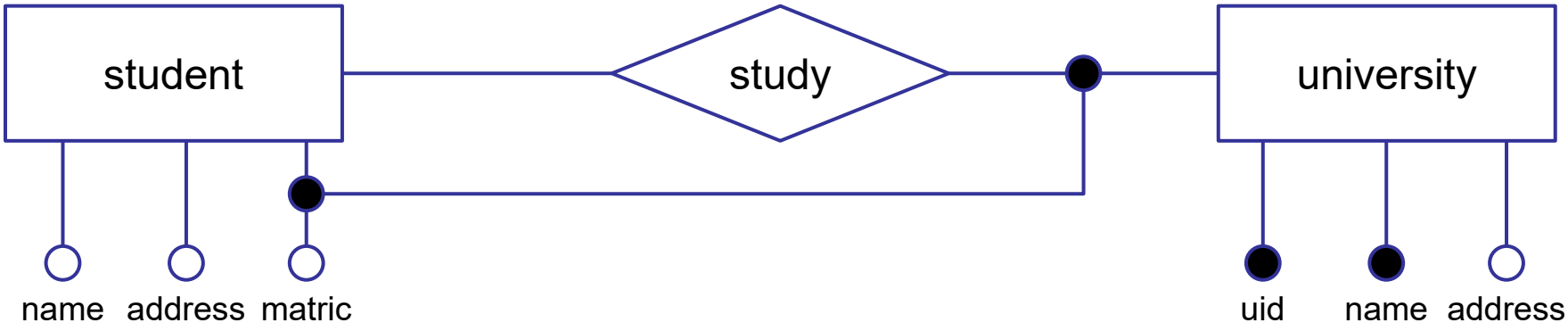
Key Attributes

- One or more attributes can uniquely **identify each entity**
- A key is a property of an **entity type**
- We use **black dots** to indicate key attributes
- A key may consist of **one attribute or multiple attributes**



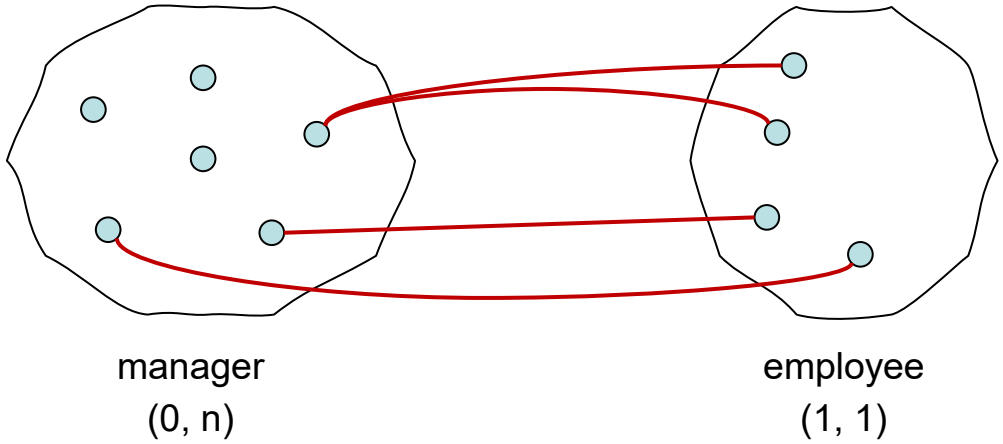
Weak Entity Types and Partial Keys

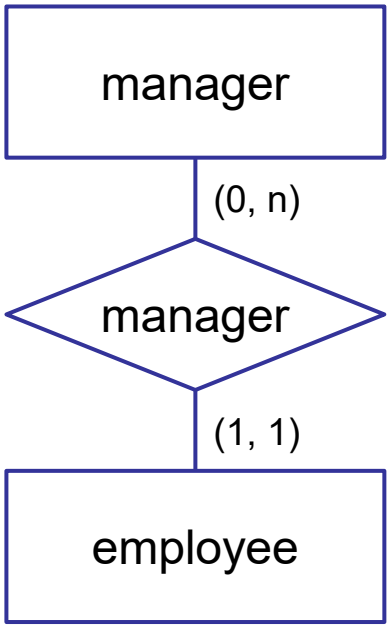
- **Some entity types can only be uniquely identified together with another entity type**
  - They can only be identified **within the scope of an identifying relationship**
- Such entity types are called **weak entity types**
- **The identifying relationship must exist** for every weak entity



Cardinality Constraints

- Cardinality constraints specify the **minimum and maximum number of relationship instances in which an entity may participate**, denoted as **(min, max)**
- **Examples: (0,1), (1,1), (0,n), (2,5)**





min = 0 → optional participation
min = 1 → mandatory participation

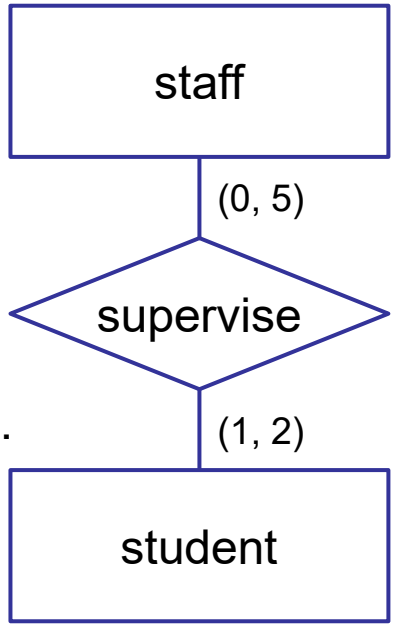
Common Cardinality Constraints

Cardinality	Name
(1, _)	characterize **mandatory participation**
(0, _)	characterize **optional participation**
(_, 1)	characterize **one-to-one relationship** if the maximum is 1 for both participating entity types characterize **one-to-many relationship** if one side has a maximum of 1 while the other side has a maximum of n
(_, n)	characterize **many-to-many relationship** if both sides have a maximum of n

Examples of Cardinality Constraints

Example #1

- Academic staff can supervise **up to** 5 research students.
- Some staff **do not** supervise students.
- Research students can have one or two supervisors.



Minimum participation: *must*, *at least*, *optional*, *may not*.
Maximum participation: *at most*, *up to*, *exactly one*, *one or more*.

Examples of Cardinality Constraints

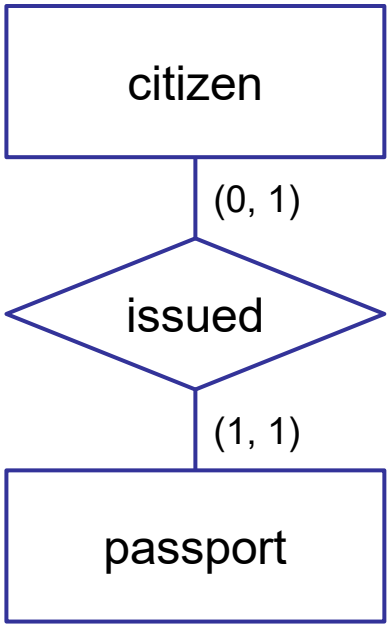
Example #2

- Assume expired or replaced passports are **removed** from the database, that is, **the database captures the current snapshot and not historical data**

What happens if we model passport information as attributes of citizen?

These attributes would have **`null`** values for citizens who do not have a passport.

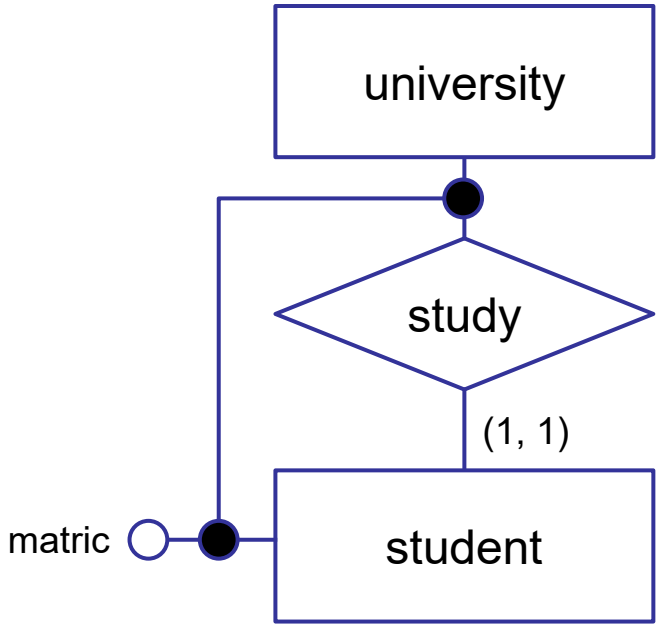
In general, we try to avoid **`null`** values.



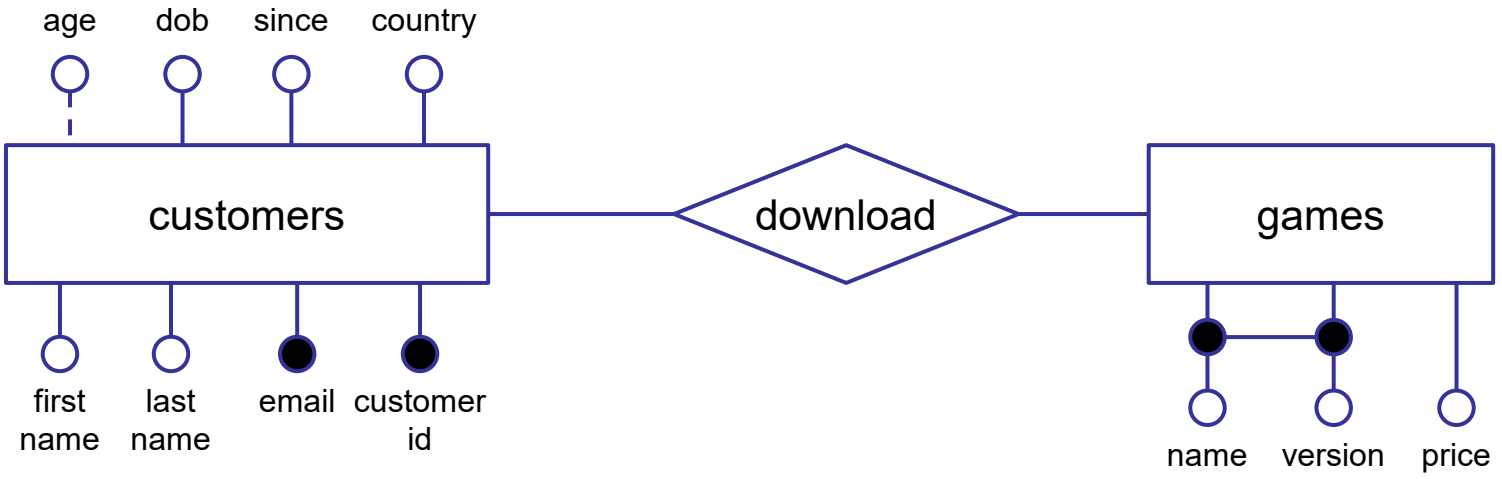
Examples of Cardinality Constraints

Example #3

- A weak entity must **participate totally** in its identifying relationship
- Therefore, its cardinality constraint is **(1, 1)**



Schema Translation



Schema Translation

Rule #1 Mapping Attributes to Domains

- Values are mapped to **domains**
- Attribute name may be changed and constraints added

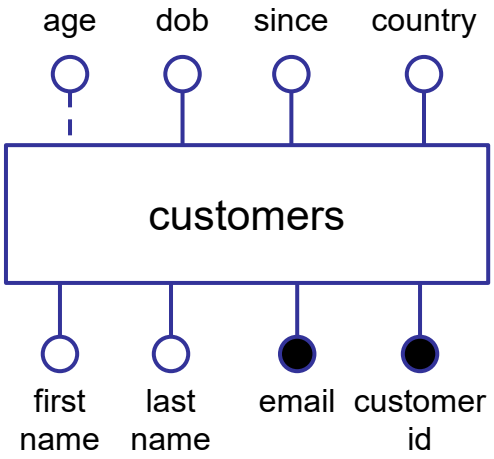



```
dob   DATE   NOT NULL
```

Schema Translation

Rule #2 Mapping Entity types to Relations

- Entity types are mapped to **relations (tables)**
- Identifying attributes are mapped to **primary key** or **candidate key**
- Derived attribute are **omitted**



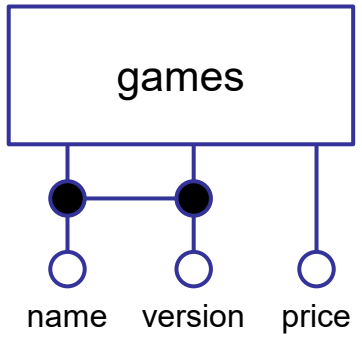

```
CREATE TABLE customers (
  email VARCHAR(256)
    UNIQUE NOT NULL,
  cid   INT   PRIMARY KEY,
  dob   DATE  NOT NULL
    :
    :
);
```

Schema Translation

Rule #2 Mapping Entity types to Relations

- Entity types are mapped to **relations/tables**
- Composite key are specified using **table constraints**

```
CREATE TABLE games (
  name      VARCHAR(256),
  version   CHAR(3),
  price     NUMERIC
    NOT NULL
    CHECK (price >= 0),
  PRIMARY KEY
    (name, version)
);
```

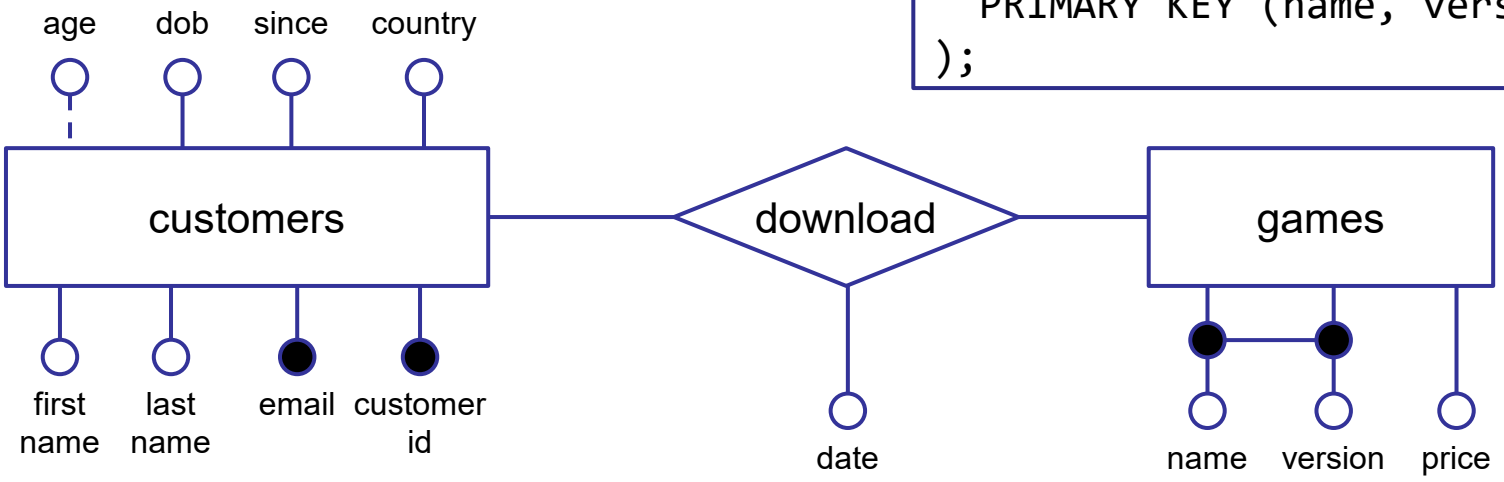


Schema Translation

Rule #3 Mapping Relationships to Relations

- Relationship types are mapped to **relations (table)**
- Primary key is formed from the primary keys of **participating entity types**

```
CREATE TABLE download (
  name    VARCHAR(256),
  version CHAR(3),
  cid     INT REFERENCES
    customers(cid),
  date    DATE  NOT NULL,
  FOREIGN KEY (name, version)
    REFERENCES games(name, version),
  PRIMARY KEY (name, version, cid)
);
```

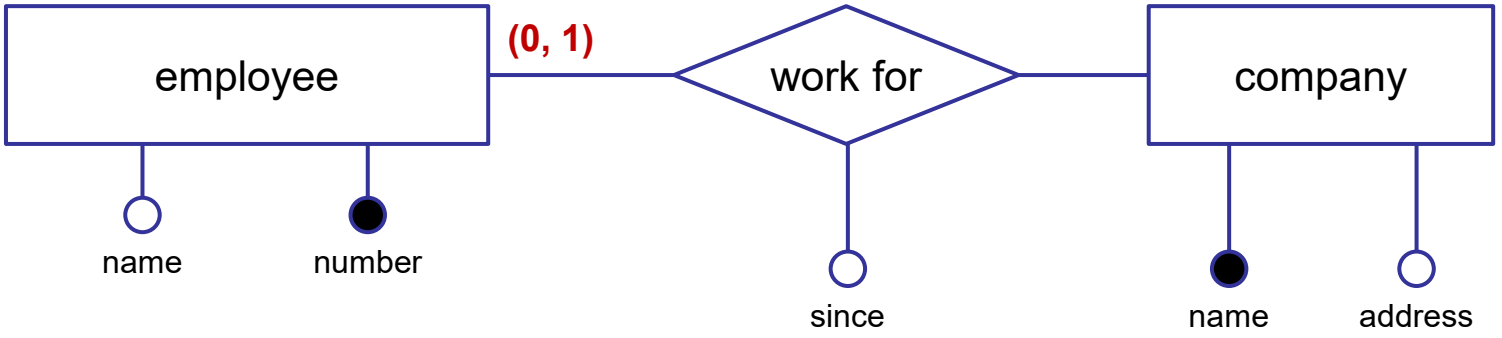


Schema Translation

Rule #4 Optimizing One-to-One and One-to-Many Relationships

- Can the **same employee** work for **two different companies**?
  - (2023-01-01, **enumber1**, cname1)
  - (2024-05-01, **enumber1**, cname2)

```
CREATE TABLE work_for (
  since    DATE  NOT NULL,
  enumber  INT
    REFERENCES employee(number),
  cname    VARCHAR(256) NOT NULL
    REFERENCES company(name),
  PRIMARY KEY(enumber)
);
```

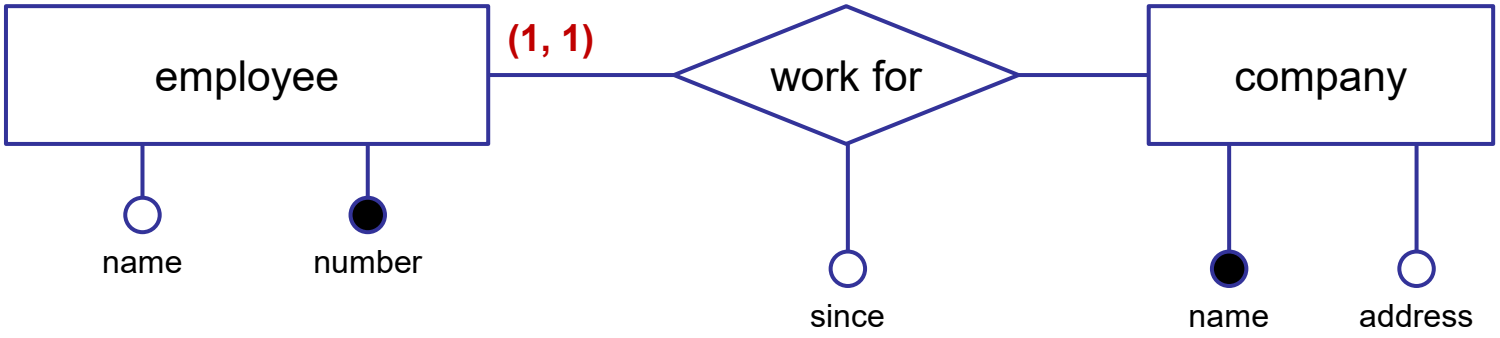


Schema Translation

Rule #5 Enforcing Total Participation

- A new employee could be inserted into **employee** without a corresponding **work_for** record

```
CREATE TABLE emp_work_for (
  since    DATE  NOT NULL,
  enumber  INT,
  ename    VARCHAR(256),
  cname    VARCHAR(256) NOT NULL
    REFERENCES company(name),
  PRIMARY KEY(enumber)
);
```

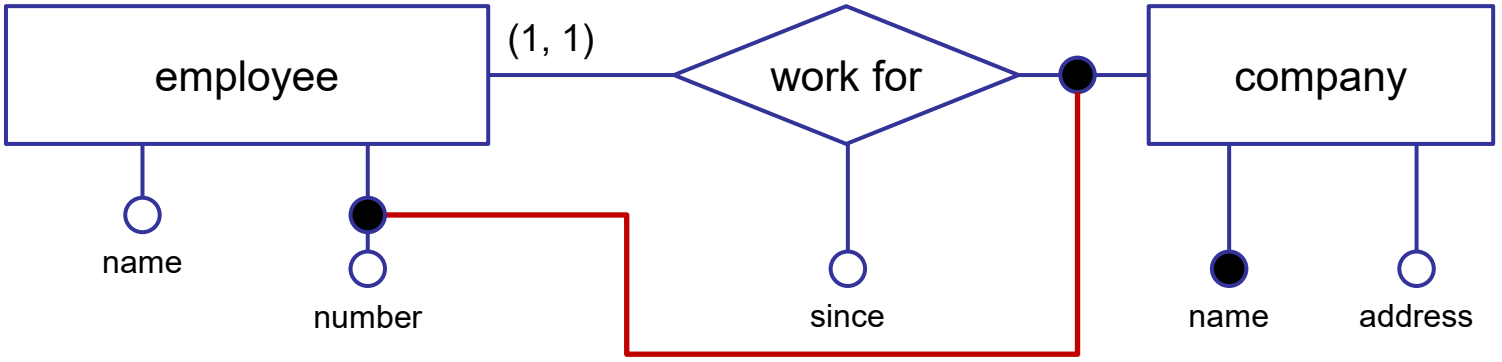


Schema Translation

Rule #6 Mapping Weak Entity Types

- Employee number is only a **partial key**. It does not uniquely identify an employee
- The primary key combines the partial key with the owner's primary key

```
CREATE TABLE emp_work_for (
  since     DATE  NOT NULL,
  enumber   INT,
  ename     VARCHAR(256),
  cname     VARCHAR(256) NOT NULL
    REFERENCES company(name),
  PRIMARY KEY(enumber, cname)
);
```



Summary

Key Concepts

- **Entity type** describes a class of objects
  - An **entity** is one instance
- **Attributes** describe properties of entities
- **Relationships** associate two or more entity types
- **Keys** uniquely identify entities
- **Weak entities** depend on another entity for identification

Summary

Modelling Constraints

- **Cardinality constraints** specify the minimum and maximum number of relationships an entity can participate in
- **Participation constraints** specify whether participation is mandatory or optional
- Good ER models capture both **data** and **business rules**

Summary

Schema Translation

- Entity type → Relation (table)
- Relationship → Relation or Foreign Key
- Attribute → Column
- Key attribute → Primary Key
- Weak entity → Table with owner's key + partial key